

ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC SÀI GÒN

KHOA TOÁN- ỨNG DỤNG



BÀI BÁO CÁO

MÔN: KHAI PHÁ DỮ LIỆU

Đề tài: XÁC ĐỊNH TẬP PHỔ BIẾN VỚI GIẢI THUẬT APRIORI

Giảng viên hướng dẫn : Đỗ Như Tài

Sinh viên thực hiện : Họ và tên

Mã số sinh viên

Phạm Ngọc Phương Uyên

3123580061

Trần Thu Hiền

3123580013

Nguyễn Châu Nhật Tường

3123580060

Lớp : DDU1231

Thành phố Hồ Chí Minh, tháng 12 năm 2025

LỜI CẢM ƠN

Trước tiên, nhóm sinh viên chúng em xin gửi lời cảm ơn chân thành và sâu sắc đến thầy Đỗ Như Tài – giảng viên phụ trách môn Khai phá dữ liệu, thầy đã tận tình giảng dạy, hướng dẫn và hỗ trợ chúng em xuyên suốt quá trình học tập cũng như trong quá trình thực hiện bài báo cáo này. Những kiến thức, định hướng và góp ý từ thầy là nền tảng quan trọng giúp nhóm chúng em hoàn thiện bài báo cáo một cách hiệu quả và thực tế hơn.

Chúng em cũng xin trân trọng cảm ơn toàn thể quý thầy cô Khoa Toán – Ứng dụng, đã tạo điều kiện thuận lợi về môi trường học tập, tài liệu và cơ sở vật chất để chúng em có thể phát huy khả năng nghiên cứu, rèn luyện tư duy và kỹ năng làm việc nhóm.

Bên cạnh đó, nhóm cũng xin ghi nhận và cảm ơn sự hỗ trợ, phối hợp chặt chẽ giữa các thành viên trong nhóm trong suốt thời gian làm việc. Mỗi thành viên đều có sự đóng góp tích cực về nội dung, kỹ thuật và hình thức trình bày, góp phần tạo nên một bài đồ án hoàn chỉnh như hiện tại.

Mặc dù đã nỗ lực rất nhiều trong quá trình thực hiện, nhưng với kiến thức và kinh nghiệm còn hạn chế, bài báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý, nhận xét quý báu từ thầy và các bạn để có thể tiếp tục hoàn thiện trong các đề tài, báo cáo sau này.

Một lần nữa, chúng em xin chân thành cảm ơn!

BẢNG PHÂN CHIA CÔNG VIỆC

Thứ tự	Họ và tên	Công việc phụ trách	Mức độ hoàn thành
1	Phạm Ngọc Phương Uyên	- Hoàn thành nội dung + code	100%
2	Trần Thu Hiền	- Hoàn thành nội dung + code	100%
3	Nguyễn Châu Nhật Tường	- Hoàn thành nội dung + code	100%

MỤC LỤC

I. Tổng Quan Về Khai Phá Luật Kết Hợp.....	6
II. Cơ Sở Lý Thuyết Của Thuật Toán Apriori	6
1. Nguyên lý Apriori và Cơ chế hoạt động.....	6
2. Các Độ Đo Cốt Lõi: Support, Confidence và Lift.....	7
2.1.1 Độ Hỗ Trợ (Support)	8
2.1.2 Độ Tin Cây (Confidence)	8
2.1.3 Độ Nâng (Lift).....	8
3. So Sánh Thuật Toán: Apriori và FP-Growth	9
4. Triển khai giải thuật Apriori bằng python (thư viện mlxtend hoặc apyori)	10
5. Tiền xử lý dữ liệu giao dịch (transactional data) trong Python trước khi áp dụng Apriori	11
6. Hàm tính toán tập phổ biến và luật kết hợp từ giải thuật Apriori trong Python.....	11
7. Điều chỉnh ngưỡng support và confidence trong Apriori.....	12
8. Trực quan hóa các luật kết hợp (association rules) dưới dạng biểu đồ	13
8.1 Biểu đồ mạng (Network Graph): được sử dụng để mô hình hóa mối quan hệ đa chiều giữa các sản phẩm.	13
8.2 Biểu đồ nhiệt (Heatmap): Heatmap cung cấp cái nhìn tổng quan dạng ma trận về cường độ liên kết giữa các cặp sản phẩm. Trực tung và trực hoành lần lượt là Tiền đề và Hệ quả, màu sắc ô giao nhau thể hiện độ mạnh của luật (Lift).	14
III. Tìm tập phổ biến từ dữ liệu mua hàng tại siêu thị.	15
1. Tổng quan về thư viện Apyori.....	15
2. Bộ dữ liệu mua hàng.....	16

2. Tiền Xử Lý Dữ Liệu	17
3. Tìm tập phổ biến.....	18
4. Trục quan kết Quả	19
IV. XÁC ĐỊNH LUẬT KẾT HỢP TỪ TẬP PHỔ BIẾN	22
1. Ôn tập lý thuyết.....	22
2. Bài tập được lấy từ Market_Basket_Optimisation.csv.....	25

XÁC ĐỊNH TẬP PHỔ BIẾN VỚI GIẢI THUẬT APRIORI

I. Tổng Quan Về Khai Phá Luật Kết Hợp

Trong kỷ nguyên dữ liệu lớn (Big Data), khả năng trích xuất các mẫu hình tiềm ẩn từ các cơ sở dữ liệu giao dịch khổng lồ đã trở thành một lợi thế cạnh tranh cốt lõi cho các doanh nghiệp, đặc biệt là trong lĩnh vực bán lẻ và thương mại điện tử. Khai phá luật kết hợp (Association Rule Mining - ARM) là một trong những kỹ thuật khai phá dữ liệu quan trọng nhất, được thiết kế để phát hiện các mối quan hệ thú vị, các mẫu hình xuất hiện đồng thời, hoặc các cấu trúc nhân quả giữa các tập hợp mục (items) trong cơ sở dữ liệu giao dịch.

Mục tiêu chính của phương pháp này là xác định các quy tắc "Nếu-Thì" (If-Then rules), cho phép dự đoán sự xuất hiện của một mục dựa trên sự hiện diện của các mục khác trong cùng một giao dịch.

Báo cáo này tập trung vào việc nghiên cứu thuật toán Apriori, một giải thuật nền tảng được giới thiệu bởi R. Agrawal và R. Srikant vào năm 1994, nhằm giải quyết bài toán tìm tập phổ biến (frequent itemsets).

II. Cơ Sở Lý Thuyết Của Thuật Toán Apriori

Thuật toán không chỉ đơn thuần là đếm số lượng, mà nó hoạt động dựa trên các nguyên lý xác suất để lọc nhiễu và xác định các mối quan hệ có ý nghĩa thống kê.

1. Nguyên lý Apriori và Cơ chế hoạt động

Vấn đề lớn nhất trong khai phá luật kết hợp là sự bùng nổ tổ hợp (combinatorial explosion). Với d mặt hàng, tổng số tập con có thể tạo ra là $2^d - 1$. Việc quét cơ sở dữ

liệu để tính Support cho tất cả các tổ hợp này là bất khả thi về mặt tính toán. Thuật toán Apriori giải quyết vấn đề này thông qua Tính chất Apriori (Apriori Property) hay còn gọi là tính chất chống đơn điệu (anti-monotonicity).

Định lý: Nếu một tập là phổ biến (frequent), thì tất cả các tập con không rỗng của nó cũng phải là phổ biến. Ngược lại, nếu một tập là không phổ biến (infrequent), thì tất cả các tập cha (supersets) chứa nó cũng phải là không phổ biến.

Dựa trên nguyên lý này, thuật toán hoạt động theo quy trình lặp "theo chiều rộng" (level-wise search) gồm hai bước chính: Kết nối (Join) và Cắt tia (Prune).

1. Bước Khởi tạo: Quét cơ sở dữ liệu để đếm tần suất của các mục đơn lẻ (1-itemsets). So sánh với min_sup để xác định tập phổ biến L_1
2. Bước Lặp (k):
 - Kết nối (Join): Tạo ra các tập ứng viên C_k bằng cách kết hợp các tập phổ biến L_{k-1} với chính nó.
 - Cắt tia (Prune): Sử dụng tính chất Apriori để loại bỏ các ứng viên trong C_k mà có chứa tập con (k-1) không nằm trong L_{k-1} . Điều này giúp giảm đáng kể không gian tìm kiếm trước khi thực hiện việc đếm tần suất.
 - Đếm: Quét cơ sở dữ liệu để xác định Support thực tế của các ứng viên còn lại và lọc ra L_k .

3. Kết thúc: Thuật toán dừng khi không còn tập phổ biến nào được tìm thấy.

2. Các Độ Đo Cốt Lõi: Support, Confidence và Lift

Một luật kết hợp thường được biểu diễn dưới dạng $X \Rightarrow Y$, trong đó:

- X là phần tiền đề (antecedent),
- Y là phần hệ quả (consequent).

Cả X và Y đều là các tập hợp mục không giao nhau ($X \cap Y = \emptyset$).

Để đánh giá chất lượng của một luật, ba chỉ số chính được sử dụng: Độ hỗ trợ (Support), Độ tin cậy (Confidence), và Độ nâng (Lift).

2.1.1 Độ Hỗ Trợ (Support)

Độ hỗ trợ là thước đo tần suất xuất hiện của một tập mục trong cơ sở dữ liệu. Nó đóng vai trò là bộ lọc đầu tiên để loại bỏ các mối quan hệ ngẫu nhiên hoặc quá hiếm gặp để có giá trị kinh doanh. Về mặt toán học, Support của một tập mục A được định nghĩa là tỷ lệ phần trăm các giao dịch trong cơ sở dữ liệu D có chứa A:

$$\text{Support}(A) = P(A) = \frac{\text{Số lượng giao dịch chứa } A}{\text{Tổng số lượng giao dịch } |D|}$$

Trong ngữ cảnh của thuật toán Apriori, một tham số quan trọng là "Ngưỡng hỗ trợ tối thiểu" (Minimum Support Threshold - min_sup). Chỉ những tập có giá trị Support lớn hơn hoặc bằng ngưỡng này mới được coi là "tập phổ biến" (frequent itemsets) và được giữ lại cho các bước xử lý tiếp theo. Việc xác định min_sup là một bài toán cân bằng.

2.1.2 Độ Tin Cậy (Confidence)

Độ tin cậy đo lường xác suất có điều kiện để mục Y xuất hiện trong một giao dịch khi biết rằng mục X đã xuất hiện. Đây là thước đo về độ mạnh của mối quan hệ chiều $X \Rightarrow Y$.

$$\text{Confidence}(X \Rightarrow Y) = P(Y|X) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Chỉ số này trả lời cho câu hỏi: "Trong số các khách hàng mua X, có bao nhiêu phần trăm cũng mua Y?". Tuy nhiên, Confidence có một nhược điểm chí mạng là nó không tính đến độ phổ biến tự nhiên của Y. Nếu Y là một mặt hàng rất phổ biến (ví dụ: sữa), thì $P(Y|X)$ có thể cao đơn giản vì $P(Y)$ cao, chứ không phải vì có mối liên hệ đặc biệt với X. Điều này dẫn đến các luật sai lệch (misleading rules).

2.1.3 Độ Nâng (Lift)

Để khắc phục hạn chế của Confidence, chỉ số Lift được sử dụng để đo lường mức độ tương quan thực sự giữa X và Y bằng cách so sánh xác suất xuất hiện đồng thời thực tế với xác suất xuất hiện đồng thời nếu hai mục này độc lập thống kê.

$$\text{Lift}(X \Rightarrow Y) = \frac{\text{Confidence}(X \Rightarrow Y)}{\text{Support}(Y)} = \frac{P(X \cap Y)}{P(X) \cdot P(Y)}$$

Việc phân tích giá trị Lift cung cấp cái nhìn sâu sắc về bản chất mối quan hệ ⁸:

- Lift = 1: X và Y độc lập. Sự xuất hiện của X không ảnh hưởng đến Y.
- Lift > 1: Tương quan dương. Sự hiện diện của X làm tăng khả năng xuất hiện của Y. Đây là các luật kết hợp có giá trị nhất.
- Lift < 1: Tương quan âm. Sự hiện diện của X làm giảm khả năng xuất hiện của Y (ví dụ: hai sản phẩm thay thế nhau như Pepsi và Coca-Cola).

3. So Sánh Thuật Toán: Apriori và FP-Growth

Tóm tắt Sự khác biệt chính:

Đặc điểm	Apriori	FP-Growth
Cách tiếp cận	Tạo và kiểm tra ứng cử viên (Candidate Generation and Test).	Khai thác trực tiếp (Direct Pattern Mining) bằng cách sử dụng cấu trúc dữ liệu nén.
Cơ chế chính	Sử dụng nguyên lý chống đơn điệu (Apriori Principle) để cắt tỉa không gian tìm kiếm.	Sử dụng cây FP-tree để biểu diễn dữ liệu nén và đệ quy tìm kiếm.
Quét cơ sở dữ liệu	Nhiều lần quét qua cơ sở dữ liệu giao dịch.	Thường chỉ cần hai lần quét để xây dựng cây FP-tree (không được nêu trực tiếp nhưng là đặc điểm chính của thuật toán này so với Apriori).
Biểu diễn dữ liệu	Bố cục dữ liệu chiều ngang (Horizontal Data Layout) (theo cách nó quét dữ liệu giao dịch).	Biểu diễn dạng cây nén (FP-tree).

Tóm lại, Apriori giảm thiểu số lượng ứng cử viên cần đếm bằng cách cắt tĩa dựa trên *support* (độ phổ biến), trong khi FP-Growth giảm thiểu số lần quét cơ sở dữ liệu bằng cách sử dụng một cấu trúc dữ liệu nén để lưu trữ thông tin về độ phổ biến.

4. Triển khai giải thuật Apriori bằng python (thư viện mlxtend hoặc apyori)

Quá trình thực hiện bao gồm các bước: Nhập thư viện, chuẩn bị dữ liệu, mã hóa dữ liệu, và chạy thuật toán.

```
# 1. Import thư viện [cite: 38, 39]

import pandas as pd

from mlxtend.preprocessing import TransactionEncoder

from mlxtend.frequent_patterns import apriori

# 2. Chuẩn bị dữ liệu (Danh sách các giao dịch) [cite: 114]

dataset = [['milk', 'bread'], ['milk', 'diapers', 'bread'], ['milk',
'bread']]

# 3. Mã hóa dữ liệu (Preprocessing)

te = TransactionEncoder()

te_ary = te.fit(dataset).transform(dataset)

df = pd.DataFrame(te_ary, columns=te.columns_)

# 4. Tìm tập phổ biến với Apriori [cite: 143]

frequent_itemsets = apriori(df, min_support=0.5, use_colnames=True)

print(frequent_itemsets)
```

5. Tiền xử lý dữ liệu giao dịch (transactional data) trong Python trước khi áp dụng Apriori

Tiền xử lý dữ liệu giao dịch cần đảm bảo rằng thuật toán Apriori có thể thực hiện việc đếm tần suất hỗ trợ (support count) của các tập mục ứng cử viên:

- Định dạng chuẩn hóa: Dữ liệu phải được chuẩn hóa về định dạng dễ dàng cho việc quét và đếm tần suất xuất hiện của các tập mục (ví dụ: ma trận nhị phân 0-1),.
- Xác định giới hạn: Việc tiền xử lý cũng bao gồm việc xác định ngưỡng hỗ trợ tối thiểu (minsup threshold), vì ngưỡng này ảnh hưởng đến số lượng tập mục phổ biến được tìm thấy và độ phức tạp tính toán,. Nếu ngưỡng được đặt quá thấp, quá trình tính toán có thể trở nên quá đắt đỏ

Các bước tiền xử lý:

- Thu thập các mục duy nhất (Unique Items): Xác định tất cả các mục khác nhau có trong toàn bộ cơ sở dữ liệu giao dịch.
- Tạo Ma trận Mã hóa Một lần (One-Hot Encoding Matrix): Chuyển đổi mỗi giao dịch từ danh sách các mục sang một hàng trong ma trận nhị phân 0-1, nơi mỗi cột là một mục (item).

6. Hàm tính toán tập phổ biến và luật kết hợp từ giải thuật Apriori trong Python

- Tìm tập phổ biến: Sử dụng hàm `apriori()` từ `mlxtend.frequent_patterns`.
- Tìm luật kết hợp: Sử dụng hàm `association_rules()` từ `mlxtend.frequent_patterns`.

Code mẫu:

```
from mlxtend.frequent_patterns import apriori, association_rules

# Tìm tập phổ biến [cite: 151]

frequent_itemsets = apriori(df, min_support=0.01, use_colnames=True)

# Tạo luật kết hợp [cite: 274]

rules = association_rules(frequent_itemsets, metric="confidence",
min_threshold=0.6)

print(rules.head())
```

7. Điều chỉnh ngưỡng support và confidence trong Apriori

Việc điều chỉnh các ngưỡng support (hỗ trợ) và confidence (độ tin cậy) trong thuật toán Apriori là hoàn toàn cần thiết và ảnh hưởng trực tiếp đến kết quả cuối cùng, bao gồm cả hiệu suất tính toán và chất lượng của các luật kết hợp (association rules) được tìm thấy.

- Đối với Support: Tham số này đóng vai trò bộ lọc đầu vào (pruning).
 - Nếu ngưỡng quá thấp: Số lượng tập phổ biến tăng đột biến theo hàm mũ, dẫn đến chi phí tính toán lớn và sinh ra nhiều luật nhiễu (không có ý nghĩa thống kê).
 - Nếu ngưỡng quá cao: Có nguy cơ bỏ sót các "luật hiếm" (rare items) nhưng mang lại giá trị kinh tế cao (ví dụ: các sản phẩm cao cấp ít người mua nhưng thường đi kèm phụ kiện đắt tiền).
- Đối với Confidence: Tham số này đo lường độ mạnh của luật sinh ra. Điều chỉnh confidence giúp loại bỏ các luật có mối liên hệ lỏng lẻo, đảm bảo rằng khi tiền đề (antecedent) xảy ra thì hệ quả (consequent) cũng có xác suất xảy ra cao tương ứng.

Cách thực hiện: tuân theo phương pháp thử - sai (trial-and-error) có định hướng, sử dụng thư viện mlxtend.

- Bước 1: Điều chỉnh Support (Giai đoạn tìm tập phổ biến): Sử dụng hàm `apriori` và thay đổi tham số `min_support`.

- *Cú pháp:* `frequent_itemsets = apriori(df, min_support=0.05, use_colnames=True)`

- *Chiến lược:* Bắt đầu với `min_support` cao (ví dụ: 0.1 hoặc 10%) để nắm bắt các xu hướng chính. Sau đó giảm dần (xuống 0.01 hoặc thấp hơn) để mở rộng phạm vi tìm kiếm cho đến khi số lượng tập phổ biến đạt mức đủ lớn nhưng vẫn trong khả năng xử lý của phần cứng.

- Bước 2: Điều chỉnh Confidence (Giai đoạn sinh luật): Sử dụng hàm `association_rules` và thay đổi tham số `min_threshold` với `metric="confidence"`.

- *Cú pháp:* `rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.6)`

- *Chiến lược:* Thiết lập ngưỡng `min_threshold` ở mức trung bình khá (thường từ 0.5 đến 0.7). Có thể kết hợp lọc thêm bằng chỉ số Lift (> 1) để đảm bảo mối quan hệ giữa hai vế là tích cực và không ngẫu nhiên.

8. Trực quan hóa các luật kết hợp (association rules) dưới dạng biểu đồ

8.1 Biểu đồ mạng (Network Graph): được sử dụng để mô hình hóa mối quan hệ đa chiều giữa các sản phẩm.

Trong đó:

- Nút (Node): Đại diện cho các sản phẩm (items).

- Cạnh (Edge): Đại diện cho luật kết hợp (chiều mũi tên từ Tiền đề → Hệ quả).
- Thuộc tính: Độ dày hoặc màu sắc của cạnh có thể biểu diễn giá trị của *Lift* hoặc *Confidence*.
- Thực hiện trong Python: Sử dụng thư viện NetworkX kết hợp với Matplotlib. Cần chuyển đổi DataFrame chứa luật thành danh sách các cạnh có trọng số.

Code mẫu:

```
import networkx as nx
import matplotlib.pyplot as plt
# Khởi tạo đồ thị có hướng
G = nx.DiGraph()
# Giới hạn số lượng luật để biểu đồ không bị rối (ví dụ: 50 luật mạnh nhất)
Top_rules = rules.sort_values('lift', ascending=False).head(50)
for i, row in top_rules.iterrows():
    # Thêm cạnh nối từ Antecedents sang Consequents
    G.add_edge(list(row['antecedents'])[0],
list(row['consequents'])[0], weight=row['lift'])
# Vẽ đồ thị
pos = nx.spring_layout(G, k=1)
nx.draw(G, pos, with_labels=True, node_size=1500,
node_color="skyblue", font_size=10)
plt.title("Biểu đồ mạng các luật kết hợp (Top 50 theo Lift)")
plt.show()
```

8.2 Biểu đồ nhiệt (Heatmap): Heatmap cung cấp cái nhìn tổng quan dạng ma trận về cường độ liên kết giữa các cặp sản phẩm. Trục tung và trục hoành lần lượt là Tiền đề và Hệ quả, màu sắc ô giao nhau thể hiện độ mạnh của luật (Lift).

- Thực hiện trong Python: Sử dụng thư viện Seaborn. Cần biến đổi dữ liệu về dạng ma trận (Pivot table) trước khi vẽ.

Code mẫu:

```
import seaborn as sns

# Chuyển đổi dữ liệu luật sang dạng ma trận Pivot

# Lưu ý: Cần xử lý dữ liệu để đảm bảo mỗi ô chỉ chứa 1 giá trị duy nhất

pivot = rules.pivot(index='antecedents', columns='consequents',
values='lift')

# Vẽ Heatmap

plt.figure(figsize=(10, 8))

sns.heatmap(pivot, annot=True, cmap="YlGnBu", fmt=".2f")

plt.title("Ma trận cường độ liên kết (Lift) giữa các sản phẩm")

plt.ylabel("Tiền đề (Antecedents)") plt.xlabel("Hệ quả (Consequents)")

plt.show()
```

III. Tìm tập phổ biến từ dữ liệu mua hàng tại siêu thị.

1. Tổng quan về thư viện Apyori

Khác với thư viện mlxtend đã thực hành ở phần trước (yêu cầu dữ liệu đầu vào phải ở dạng *One-Hot Encoded* thông qua TransactionEncoder), thư viện apyori chấp nhận dữ liệu đầu vào đơn giản hơn là một danh sách các giao dịch (List of Lists). Điều này giúp giảm bớt bước mã hóa dữ liệu phức tạp, đặc biệt hữu ích khi làm việc với dữ liệu dạng thưa (sparse data).

2. Bộ dữ liệu mua hàng

- 5 dòng đầu:

```
df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)
df.head()
```

✓ 0.1s [Open 'df' in Data Wrangler](#)

	0	1	2	3	4	5
0	shrimp	almonds	avocado	vegetables mix	green grapes	whole weat flour
1	burgers	meatballs	eggs	Missing value	Missing value	Missing value
2	chutney	Missing value	Missing value	Missing value	Missing value	Missing value
3	turkey	avocado	Missing value	Missing value	Missing value	Missing value
4	mineral water	milk	energy bar	whole wheat rice	green tea	Missing value

- Thông tin dữ liệu

```
df.info()
```

✓ 0.3s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7501 entries, 0 to 7500
Data columns (total 20 columns):
#   Column  Non-Null Count  Dtype
---  -----  -
0    0      7501 non-null    object
1    1      7501 non-null    object
2    2      7501 non-null    object
3    3      7501 non-null    object
4    4      7501 non-null    object
5    5      7501 non-null    object
6    6      7501 non-null    object
7    7      7501 non-null    object
8    8      7501 non-null    object
9    9      7501 non-null    object
10   10     7501 non-null    object
11   11     7501 non-null    object
12   12     7501 non-null    object
13   13     7501 non-null    object
14   14     7501 non-null    object
15   15     7501 non-null    object
16   16     7501 non-null    object
17   17     7501 non-null    object
18   18     7501 non-null    object
19   19     7501 non-null    object
dtypes: object(20)
memory usage: 1.1+ MB
```

Nhận Xét:

- Quy mô dữ liệu: Bộ dữ liệu bao gồm 7.501 giao dịch (dòng), đây là kích thước mẫu đủ lớn để đảm bảo độ tin cậy thống kê khi khai phá luật kết hợp.
- Cấu trúc giỏ hàng: Mỗi giao dịch chứa tối đa 20 sản phẩm (tương ứng 20 cột). Tuy nhiên, độ dài các giao dịch không đồng đều (có người mua 1 món, có người mua 20 món).
- Đặc điểm phân bố: Dữ liệu chứa nhiều giá trị NaN (Missing Values), đặc biệt ở các cột phía sau. Đây là đặc thù bình thường của dữ liệu giao dịch bán lẻ (dữ liệu dạng thưa).

2. Tiền Xử Lý Dữ Liệu

Bước 1: Xử lý các giá trị NaN

```
# Thay thế giá trị rỗng bằng 0.
df.fillna(0,inplace=True)
df.head()
```

✓ 0.0s Open 'df' in Data Wrangler

	0	1	2	3
0	shrimp	almonds	avocado	vegetables mix
1	burgers	meatballs	eggs	0
2	chutney	0	0	0
3	turkey	avocado	0	0
4	mineral water	milk	energy bar	whole wheat rice

Bước 2: Chuyển đổi dữ liệu

```
# Chuyển đổi dữ liệu sang dạng list of lists

transactions = []

for i in range(0,len(df)):
    transactions.append([str(df.values[i,j])
    for j in range(0,20)
    if str(df.values[i,j])!='0'])

print(f"\nTổng transactions: {len(transactions)}")
print(f"Mẫu transactions: {transactions[0:5]}")
```

✓ 0.1s

Tổng transactions: 7501

Mẫu transactions: [['shrimp', 'almonds', 'avocado', 'vegetables mix', 'green grapes', 'whole weat flour', 'yams', 'cot'

Bước 3: Hiện thị thống kê

```
# Thống kê số items trong mỗi transaction
transaction_lengths = [len(t) for t in transactions]
print(f"\nThống kê độ dài Transaction:")
print(f"Min items: {min(transaction_lengths)}")
print(f"Max items: {max(transaction_lengths)}")
print(f"Average items: {np.mean(transaction_lengths):.2f}")
```

✓ 0.0s

Thống kê độ dài Transaction:

Min items: 1

Max items: 20

Average items: 3.91

3. Tìm tập phổ biến

Áp dụng thuật toán apriori để tìm luật kết hợp:

- Thiết lập các siêu tham số (Hyperparameters):
 - min_support: Chọn 0.003. (Ví dụ: Sản phẩm được mua 3 lần/ngày * 7 ngày = 21 lần / 7500 giao dịch ≈ 0.003)
 - min_confidence: Chọn 0.2. (Luật phải đúng ít nhất 20% số trường hợp)
 - min_lift: Chọn 3. (Chỉ quan tâm đến các mối quan hệ tương quan mạnh, gấp 3 lần ngẫu nhiên)
 - min_length: 2. (Luật phải có ít nhất 2 sản phẩm: A -> B)

```
rules = apriori(
    transactions,
    min_support=0.003,
    min_confidence=0.2,
    min_lift=3,
    min_length=2,
    max_length=2 # Giới hạn ở luật 2 thành phần để dễ phân tích ban đầu
)
```

```
# Chuyển đổi kết quả từ generator sang list để xử lý
results = list(rules)
print(f"Số lượng luật kết hợp tìm thấy: {len(results)}")
```

✓ 0.0s

Số lượng luật kết hợp tìm thấy: 9

- Chuyển đổi kết quả thành Data Frame: vì kết quả trả về từ apyori là một cấu trúc lồng ghép (nested object) khó đọc (RelationRecord) và để báo cáo trực quan và dễ phân tích.
 - result: frozenset chứa các item trong luật
 - result: giá trị support của luật
 - result: danh sách ordered_statistics

```
# Chuyển đổi kết quả thành DataFrame
def inspect_apyori_results(results):
    lhs = []
    rhs = []
    supports = []
    confidences = []
    lifts = []

    for result in results:
        support_val = result.support
        ordered_stats = result.ordered_statistics

        for stat in ordered_stats:
            # stat: items_base (tiền đề)
            # stat: items_add (hệ quả)
            # stat: confidence
            # stat: lift

            # Chỉ lấy các luật có tiền đề và hệ quả không rỗng
            if stat.items_base and stat.items_add:
                lhs.append(list(stat.items_base))
                rhs.append(list(stat.items_add))
                supports.append(support_val)
                confidences.append(stat.confidence)
                lifts.append(stat.lift)

    return pd.DataFrame({
        'LHs': lhs,
        'RHs': rhs,
        'Support': supports,
        'Confidence': confidences,
        'Lift': lifts
    })

# Tạo DataFrame kết quả
results_df = inspect_apyori_results(results)
```

✓ 0.0s

4. Trực quan kết Quả

```
# Sắp xếp theo Lift giảm dần để thấy các luật mạnh nhất
top_rules = results_df.nlargest(10, 'Lift')

print("\nTop 10 Luật Kết Hợp Có Độ Nâng (Lift) Cao Nhất:")
print(top_rules.to_string(index=False))
```

✓ 0.0s

Top 10 Luật Kết Hợp Có Độ Nâng (Lift) Cao Nhất:

LHs	RHs	Support	Confidence	Lift
[fromage blanc]	[honey]	0.003333	0.245098	5.164271
[light cream]	[chicken]	0.004533	0.290598	4.843951
[pasta]	[escalope]	0.005866	0.372881	4.700812
[pasta]	[shrimp]	0.005066	0.322034	4.506672
[whole wheat pasta]	[olive oil]	0.007999	0.271493	4.122410
[tomato sauce]	[ground beef]	0.005333	0.377358	3.840659
[mushroom cream sauce]	[escalope]	0.005733	0.300699	3.790833
[herb & pepper]	[ground beef]	0.015998	0.323450	3.291994
[light cream]	[olive oil]	0.003200	0.205128	3.114710

Nhận Xét:

Mối quan hệ tương quan mạnh mẽ (High Lift):

- Tất cả 10 luật hàng đầu đều có chỉ số Lift > 3, chứng tỏ mối liên hệ giữa sản phẩm ở vế trái (LHs) và vế phải (RHs) là rất chặt chẽ, không phải ngẫu nhiên.
- Nổi bật nhất: Cặp sản phẩm [fromage blanc] (phô mai tươi) → [honey] (mật ong) đứng đầu bảng với Lift ≈ 5.16 .
 - Ý nghĩa:* Một khách hàng khi đã mua phô mai tươi thì khả năng họ mua thêm mật ong cao gấp 5.16 lần so với một khách hàng mua sắm ngẫu nhiên. Đây là một cặp bài trùng rất rõ ràng trong thói quen ăn uống (ăn tráng miệng).

Sự phù hợp về mặt ngữ nghĩa (Thói quen ẩm thực): Các luật tìm được phản ánh rất chính xác thói quen nấu nướng thực tế của người phương Tây:

- Nhóm Pasta (Mỳ Ý):
 - [pasta] → [escalope] (thịt lát mỏng): Lift ≈ 4.70 .
 - [pasta] → [shrimp] (tôm): Lift ≈ 4.50 .
 - [whole wheat pasta] → [olive oil] (dầu ô liu): Lift ≈ 4.12 .
⇒ Điều này cho thấy khách hàng thường mua nguyên liệu chính (mỳ) kèm

theo nguyên liệu chế biến (dầu oliu) hoặc món ăn kèm (thịt, tôm) để hoàn thiện bữa ăn.

- Nhóm Sốt (Sauce):
 - [tomato sauce] → [ground beef] (thịt bò xay): Lift ≈ 3.84 . Đây là nguyên liệu chuẩn để làm món Mỳ ý sốt bò băm (Bolognese).

Đặc điểm về Độ hỗ trợ (Support) và Độ tin cậy (Confidence):

- Support thấp: Các giá trị Support dao động từ 0.003 (0.3%) đến 0.015 (1.5%).
 - *Nhận định:* Các luật này tuy rất mạnh (Lift cao) nhưng lại là các quy luật "ngách" (niche rules), xuất hiện trong các nhóm khách hàng cụ thể chứ không phải là hành vi đại trà (như mua Sữa hay Bánh mì).
- Confidence trung bình: Độ tin cậy dao động từ 20% - 37%.
 - Ví dụ: Luật [tomato sauce] → [ground beef] có Confidence cao nhất trong bảng ($\approx 37.7\%$). Nghĩa là cứ 10 người mua sốt cà chua thì có gần 4 người sẽ mua kèm thịt bò xay.

Kết luận:

- Về quy trình: apyori tinh gọn hơn ở khâu xử lý dữ liệu đầu vào (không cần TransactionEncoder để tạo ma trận thưa One-hot).
- Về hiệu năng: Với các tập dữ liệu cực lớn, apyori có thể chậm hơn so với mlxtend (thư viện được tối ưu hóa tốt hơn với Pandas/Numpy).
- Về kết quả đầu ra: mlxtend tách biệt hai bước "tìm tập phổ biến" và "sinh luật", giúp người dùng linh hoạt kiểm soát từng giai đoạn. Ngược lại, apyori thực hiện đồng thời và trả về kết quả gộp, đòi hỏi lập trình viên phải viết thêm mã xử lý hậu kỳ (như Bước 4 ở trên) để có được bảng dữ liệu tường minh.

IV. XÁC ĐỊNH LUẬT KẾT HỢP TỪ TẬP PHỔ BIẾN

1. Ôn tập lý thuyết

Luật kết hợp (association rules) là gì? Support, confidence và lift trong luật kết hợp

Luật kết hợp (association rules) là phương pháp khai phá dữ liệu dùng để phát hiện các mối quan hệ tiềm ẩn giữa các mục thường xuyên xuất hiện cùng nhau trong cơ sở dữ liệu giao dịch. Luật kết hợp có dạng $X \Rightarrow Y$, trong đó X và Y là hai tập mục không giao nhau, biểu thị rằng khi X xuất hiện thì Y có xu hướng xuất hiện theo.

Chỉ số support phản ánh mức độ phổ biến của luật, được tính bằng tỷ lệ số giao dịch chứa đồng thời cả X và Y trên tổng số giao dịch. Confidence thể hiện độ tin cậy của luật, là xác suất xuất hiện Y khi đã xuất hiện X . Trong khi đó, lift dùng để đánh giá mức độ phụ thuộc giữa X và Y ; nếu $\text{lift} > 1$ thì sự xuất hiện của X làm tăng khả năng xuất hiện của Y , cho thấy luật có ý nghĩa thực tế.

Quá trình tìm luật kết hợp từ tập phổ biến bao gồm những bước nào? Làm thế nào để chuyển từ tập phổ biến sang luật kết hợp?

Quá trình tìm luật kết hợp thường gồm hai giai đoạn chính. Giai đoạn thứ nhất là tìm các tập mục phổ biến (frequent itemsets), tức là các tập mục có support lớn hơn hoặc bằng ngưỡng support tối thiểu do người phân tích đặt ra. Giai đoạn này thường sử dụng các thuật toán như Apriori hoặc FP-Growth.

Giai đoạn thứ hai là sinh các luật kết hợp từ các tập mục phổ biến đã tìm được. Mỗi tập mục phổ biến sẽ được chia thành hai phần X và Y để tạo thành luật $X \Rightarrow Y$, sau đó các chỉ số confidence và lift được tính toán để đánh giá chất lượng của luật. Chỉ những luật thỏa mãn các ngưỡng đánh giá mới được giữ lại.

Khi nào một luật kết hợp được coi là "mạnh" (strong rule)? Các chỉ số như lift hoặc confidence ảnh hưởng như thế nào đến đánh giá này?

Một luật kết hợp được coi là “mạnh” khi đồng thời thỏa mãn các điều kiện về support, confidence và lift. Support đảm bảo rằng luật xuất hiện đủ nhiều trong dữ liệu, tránh các luật hiếm và không đáng tin cậy. Confidence cao cho thấy mối quan hệ giữa X và Y có độ tin cậy tốt.

Tuy nhiên, confidence cao chưa đủ để khẳng định luật có ý nghĩa, do đó cần xem xét thêm chỉ số lift. Lift lớn hơn 1 chứng tỏ mối quan hệ giữa X và Y không phải ngẫu nhiên mà có sự phụ thuộc thực sự, từ đó giúp đánh giá chính xác hơn giá trị của luật trong thực tiễn.

Viết đoạn code mẫu bằng Python (sử dụng thư viện như mlxtend hoặc apyori) để tìm luật kết hợp từ tập phổ biến không? Hãy mô tả các bước thực hiện

B1: Cài đặt thư viện

```
pip install mlxtend
```

B2: Chuẩn bị dữ liệu giao dịch

```
import pandas as pd

# Dữ liệu giao dịch mẫu
transactions = [

    ['Sữa', 'Bánh mì', 'Bơ'],

    ['Sữa', 'Bánh mì'],

    ['Bánh mì', 'Bơ'],

    ['Sữa', 'Bơ'],

    ['Sữa', 'Bánh mì', 'Bơ']

]
```

B3: Mã hóa dữ liệu

```
from mlxtend.preprocessing import TransactionEncoder

te = TransactionEncoder()

te_array = te.fit(transactions).transform(transactions)

df = pd.DataFrame(te_array, columns=te.columns_)
```

```
print(df)
```

B4: Tìm tập mục phổ biến

```
from mlxtend.frequent_patterns import apriori

frequent_itemsets = apriori(

    df,

    min_support=0.4,

    use_colnames=True

)

print(frequent_itemsets)
```

B5: Sinh luật kết hợp

```
from mlxtend.frequent_patterns import association_rules

rules = association_rules(

    frequent_itemsets,

    metric="confidence",

    min_threshold=0.6

)

# Hiển thị các luật quan trọng

print(rules[['antecedents',    'consequents',    'support',    'confidence',
'lift']])
```

B6: Lọc luật mạnh (confidence ≥ 0.6 , lift > 1)

```
strong_rules = rules[

    (rules['confidence'] >= 0.6) &

    (rules['lift'] > 1)

]

print(strong_rules[['antecedents', 'consequents', 'support', 'confidence',
'lift']])
```

2. Bài tập được lấy từ **Market_Basket_Optimisation.csv**

Bài toán 1

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules
from mlxtend.preprocessing import TransactionEncoder
import matplotlib.pyplot as plt
import seaborn as sns

# --- 1. Đọc và Tiền xử lý Dữ liệu (Lặp lại từ phân tích trước) ---
try:
    df = pd.read_csv('Market_Basket_Optimisation.csv', header=None)
except FileNotFoundError:
    print("Lỗi: Không tìm thấy tệp 'Market_Basket_Optimisation.csv'.")
    exit()

transactions = []
for i in range(len(df)):
    transaction = [str(item) for item in df.iloc[i].dropna()]
    if transaction:
        transactions.append(transaction)

te = TransactionEncoder()
te_ary = te.fit(transactions).transform(transactions)
df_ohe = pd.DataFrame(te_ary, columns=te.columns_)

# --- 2. Áp dụng Apriori và Tạo Luật Kết hợp ---
# Sử dụng Min Support thấp để có đủ luật để trực quan hóa
min_support_value = 0.003
```

```

frequent_itemsets = apriori(df_ohe, min_support=min_support_value,
use_colnames=True)

# Tạo luật kết hợp, sử dụng Min Lift = 1.0 (vì chúng ta muốn thấy tất cả các
mối quan hệ)

rules = association_rules(
    frequent_itemsets,
    metric="lift",
    min_threshold=1.0
)

# Thêm cột độ dài cho Antecedents (Số lượng item trong vế điều kiện)
rules['antecedents_length'] = rules['antecedents'].apply(lambda x: len(x))

# --- 3. Trực quan hóa Biểu đồ 1: Support vs Confidence ---

plt.figure(figsize=(10, 6))

# Biểu đồ phân tán giữa Support và Confidence, sử dụng Lift để tô màu các
điểm

# Màu sắc: Giá trị Lift càng cao (đỏ) thì luật càng mạnh và có ý nghĩa
scatter = sns.scatterplot(
    x="support",
    y="confidence",
    hue="lift", # Dùng Lift để tô màu
    data=rules,
    palette="viridis", # Bảng màu đẹp để thể hiện gradient
    alpha=0.7,
    s=70 # Kích thước điểm
)

# Thêm tiêu đề và nhãn

```

```

plt.title('Mối quan hệ giữa Support và Confidence (Tô màu theo Lift)')
plt.xlabel('Support (Hỗ trợ)')
plt.ylabel('Confidence (Độ tin cậy)')
plt.grid(axis='y', linestyle='--', alpha=0.5)

# Thêm chú giải màu sắc (colorbar) cho Lift
cbar = plt.colorbar(scatter.collections[0])
cbar.set_label('Lift (Độ nâng)')

plt.tight_layout()
plt.show()

# --- 4. Trực quan hóa Biểu đồ 2: Antecedent Length vs Confidence ---

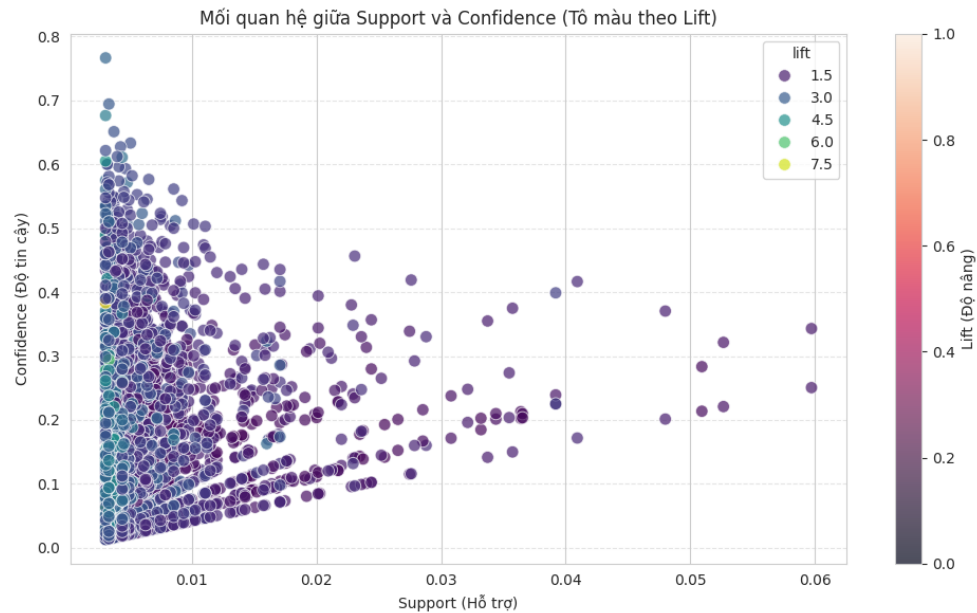
plt.figure(figsize=(10, 6))

# Boxplot hoặc Violinplot để xem phân phối Confidence theo số lượng Item
# Sử dụng boxplot để thấy rõ phân phối và giá trị ngoại lai
sns.boxplot(
    x="antecedents_length",
    y="confidence",
    data=rules[rules['antecedents_length'] < 6], # Giới hạn độ dài để biểu đồ dễ đọc
    palette="Pastell"
)

# Thêm tiêu đề và nhãn
plt.title('Phân phối Confidence theo Độ dài Antecedents (Số lượng Item)')
plt.xlabel('Antecedents Length (Số lượng Item trong vế điều kiện)')
plt.ylabel('Confidence (Độ tin cậy)')
plt.grid(axis='y', linestyle='--', alpha=0.5)

```

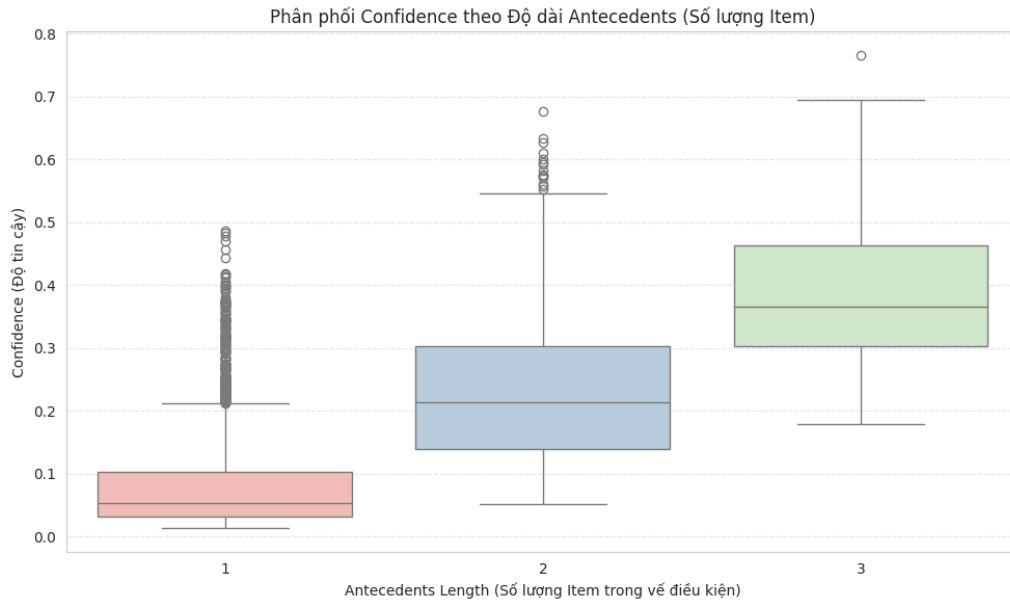
```
plt.tight_layout()
plt.show()
```



Biểu đồ Phân tán: Support vs Confidence (Tô màu theo Lift)

Tính phổ biến và Độ tin cậy: Đa số các luật có Support thấp (tập trung bên trái). Rất ít luật có cả Support và Confidence cao (Góc trên bên phải).

Giá trị Lift (Tương quan): Các luật có ý nghĩa nhất có màu vàng/đỏ (Lift cao), thường nằm rải rác. Các luật này rất quan trọng cho các chiến lược khuyến mãi chéo. Phần lớn luật có Lift gần 1 (màu xanh/tím) và ít thú vị.



Biểu đồ Hộp: Antecedent Length vs Confidence

Xu hướng Confidence: Độ tin cậy (Confidence) có xu hướng tăng khi Antecedent Length (Số lượng Item) tăng lên. Điều này là do các luật trở nên cụ thể hơn.

Độ biến động: Các luật có độ dài ngắn (Length = 1 hoặc 2) có phạm vi Confidence rộng nhất (hộp dài hơn), cho thấy sự đa dạng lớn về độ tin cậy trong các luật phổ biến.

Hạn chế: Mặc dù Confidence tăng theo độ dài, cần nhớ rằng Support giảm rất nhanh khi độ dài tăng (các luật dài rất hiếm).

Bài toán 2

```
import pandas as pd

from collections import defaultdict
from itertools import combinations
from operator import itemgetter

# Đọc dữ liệu Market Basket
data = pd.read_csv("Market_Basket_Optimisation.csv", header=None)

# Tạo danh sách giao dịch
transactions = []

for i in range(data.shape[0]):
    transactions.append(set(data.iloc[i].dropna()))
```

```
num_transactions = len(transactions)
print("Số giao dịch:", num_transactions)
```

... Số giao dịch: 7501

```
min_support = 0.03
min_confidence = 0.5
item_counts = defaultdict(int)
for transaction in transactions:
    for item in transaction:
        item_counts[item] += 1
frequent_itemsets = dict()
frequent_itemsets[1] = {
    frozenset([item]): count
    for item, count in item_counts.items()
    if count / num_transactions >= min_support
}
print("Số item phổ biến:", len(frequent_itemsets[1]))
```

Số item phổ biến: 36

```
frequent_itemsets[2] = defaultdict(int)

for transaction in transactions:
    for itemset in combinations(sorted(transaction), 2):
        frequent_itemsets[2][frozenset(itemset)] += 1

frequent_itemsets[2] = {
    itemset: count
    for itemset, count in frequent_itemsets[2].items()
    if count / num_transactions >= min_support
}

print("Số tập 2-item phổ biến:", len(frequent_itemsets[2]))
```

Số tập 2-item phổ biến: 18

```
correct_counts = defaultdict(int)
incorrect_counts = defaultdict(int)

for transaction in transactions:
    for premise, conclusion in candidate_rules:
        if premise.issubset(transaction):
            if conclusion in transaction:
                correct_counts[(premise, conclusion)] += 1
            else:
                incorrect_counts[(premise, conclusion)] += 1
rule_confidence = dict()

for rule in candidate_rules:
    correct = correct_counts[rule]
    incorrect = incorrect_counts[rule]
    if correct + incorrect > 0:
        confidence = correct / float(correct + incorrect)
        if confidence >= min_confidence:
            rule_confidence[rule] = confidence

print("Số luật thỏa confidence:", len(rule_confidence))
```

Số luật thỏa confidence: 0

```
sorted_confidence = sorted(
    rule_confidence.items(),
    key=itemgetter(1),
    reverse=True
)

top_n = min(5, len(sorted_confidence))

for i in range(top_n):
    (premise, conclusion), confidence = sorted_confidence[i]
    print("Rule #{}".format(i + 1))
    print("If a customer buys {} they will also buy {}".format(set(premise), conclusion))
    print("Confidence: {:.3f}".format(confidence))
```

```
print("")
```

NHẬN XÉT KẾT QUẢ

Kết quả thực nghiệm cho thấy, với ngưỡng $\text{support} = 0.03$ và $\text{confidence} = 0.5$, không thu được luật kết hợp nào thỏa mãn đồng thời hai điều kiện. Nguyên nhân là do dữ liệu Market Basket có tính phân tán cao, số lượng mặt hàng lớn trong khi mỗi giao dịch chỉ chứa một số ít sản phẩm.

Kết quả khai phá cho thấy việc lựa chọn ngưỡng support và confidence có ảnh hưởng lớn đến số lượng luật kết hợp thu được. Với dữ liệu Market Basket có tính đa dạng cao, việc sử dụng ngưỡng quá lớn có thể dẫn đến không phát hiện được luật. Do đó, cần điều chỉnh tham số phù hợp với đặc điểm dữ liệu để thu được các luật có ý nghĩa.